



Revisão Prova 3 ED2

1. Busca em Grafos (DFS e BFS)

A busca ou percurso em grafos consiste na examinação sistemática de vértices e arestas. Inicialmente, todas as estruturas são marcadas como não visitadas e, após o exame, passam para o status de visitadas.

1.1. Busca em Profundidade (Depth-First Search - DFS)

A DFS realiza incursões profundas no grafo, progredindo a partir do vizinho visitado mais recentemente.

Informações Cruciais

- **Aplicação:** Exploração completa de estruturas, detecção de ciclos e classificação de arestas/arcs.
- **Característica Principal:** Utiliza o mecanismo de recursividade ou uma pilha explícita para guiar o caminamento.
- **Estratégia:** Marca o vértice atual como visitado e, recursivamente, chama a função para cada vizinho que ainda não tenha sido explorado. Em grafos direcionados, classifica os arcos analisados em arcos de árvore, avanço, retorno ou cruzamento.
- **Complexidade de Tempo:** Limita-se a $\mathcal{O}(V + E)$ se o grafo for representado por listas de adjacência, pois cada vértice e cada aresta são analisados uma quantidade fixa de vezes.

Materiais de apoio:

<https://www.cs.usfca.edu/~galles/visualization/DFS.html>

https://youtu.be/Urx87-NMm6c?si=T_RS82R4qI39xZZ2

Implementação em Java

```
import java.util.*;  
  
public class BuscaProfundidade{
```

```

// Função principal que gerencia o reinício em componen
tes desconexos
public void dfsReinicie(int n, List<List<Integer>> graf
o){
    boolean[] visitados = new boolean[n];
    for (int v = 0; v < n; v++) {
        if (!visitados[v]) {
            dfs(v, grafo, visitados);
        }
    }
}

private void dfs(int v, List<List<Integer>> grafo, bool
ean[] visitados){
    visitados[v] = true;
    System.out.print(v + " "); // Ordem de visitaço

    for (int w : grafo.get(v)) {
        if (!visitados[w]) {
            dfs(w, grafo, visitados);
        }
    }
}
}

```

1.2. Busca em Largura (Breadth-First Search - BFS)

A BFS avança de maneira concêntrica (em camadas), certificando-se de explorar toda a vizinhança de um nível antes de progredir para o seguinte.

Informações Cruciais

- **Aplicação:** Determinação do caminho mínimo (menor número de arestas) em grafos não ponderados.
- **Característica Principal:** Utiliza estritamente uma estrutura de fila (Q) para guiar a ordem de exploração.
- **Estratégia:** Insere o nó raiz na fila e o marca como visitado. Enquanto a fila não estiver vazia, remove o elemento frontal e insere todos os seus vizinhos não visitados, marcando-os simultaneamente.

- **Complexidade de Tempo:** $\mathcal{O}(V + E)$ utilizando listas de adjacências.

Materiais de apoio:

<https://www.cs.usfca.edu/~galles/visualization/BFS.html>

<https://youtu.be/HZ5YTanv5QE?si=ImaDCyYpCGlihcXT>

Implementação em Java

```
import java.util.*;

public class BuscaLargura{
    public void bfs(int n, List<List<Integer>> grafo, int o
    rigem){
        boolean[] visitados = new boolean[n];
        Queue<Integer> fila = new LinkedList<>();

        visitados[origem] = true;
        fila.add(origem);

        while (!fila.isEmpty()) {
            int v = fila.poll();
            System.out.print(v + " "); // Ordem de visitaçã
o

            for (int w : grafo.get(v)) {
                if (!visitados[w]) {
                    visitados[w] = true;
                    fila.add(w);
                }
            }
        }
    }
}
```

2. Algoritmo de Dijkstra

O algoritmo de Dijkstra é uma abordagem gulosa (*greedy*) para encontrar o caminho de custo mínimo de um vértice de origem para todos os outros vértices em um grafo.

Informações Cruciais

- **Aplicação:** Problema de Caminho Mais Curto
- **Restrição Principal: Não suporta arestas com pesos negativos.** Se houver um peso negativo, o algoritmo pode retornar resultados incorretos, pois assume que, uma vez que um nó é visitado, seu custo mínimo foi definitivamente encontrado.
- **Estratégia:** Utiliza a técnica de relaxamento das arestas, sempre escolhendo o vértice não visitado com a menor distância acumulada.
- **Complexidade de Tempo:** Usando apenas arrays simples (como na implementação abaixo), a complexidade é $O(V^2)$, onde V é o número de vértices. É ideal para grafos densos.

Materiais de apoio:

https://youtu.be/EFg3u_E6eHU?si=Wxd6To-27RBAusap

<https://youtu.be/HUSolQwJ6cl?si=k7-brmqjk0X0fuE2>

<https://www.cs.usfca.edu/~galles/visualization/Dijkstra.html>

Implementação em Java

Implementação utilizando matriz de adjacências .

```
public class Dijkstra{
    public static int[] Dijkstra(int[][] matrizPesos) {
        int n = matrizPesos.length;
        int count = n;
        long[] dist = new long[n];
        int[] rota = new int[n];
        dist[0] = 0;
        rota[0] = -1;

        for (int i = 1; i < n; i++) {
            dist[i] = Integer.MAX_VALUE;
            rota[i] = -1;
        }

        boolean[] visitados = new boolean[n];
```

```

        while (count > 0) {
            count--;
            long min = Integer.MAX_VALUE;
            int v = -1;
            for (int i = 0; i < n; i++) {
                if (!vizitados[i] && dist[i] != Integer.MAX
VALUE && dist[i] < min) {
                    min = dist[i];
                    v = i;
                }
            }

            if (v == -1)
                break;

            vizitados[v] = true;
            for (int u = 0; u < n; u++) {
                if (matrizPesos[v][u] > 0 && !vizitados[u])
                {
                    if (dist[v] + matrizPesos[v][u] < dist
[u]) {
                        dist[u] = dist[v] + matrizPesos[v]
[u];
                        rota[u] = v;
                    }
                }
            }

            return rota;
        }

        public static void imprimirCaminho(int[] rota, int orig
em, int destino) {

            if (rota[destino] == -1 && destino != origem) {
                System.out.println("Não existe caminho de " + o
rigem + " até " + destino);
            }
        }
    }
}

```

```

        return;
    }

    List<Integer> caminho = new ArrayList<>();
    int atual = destino;

    while (atual != -1) {
        caminho.add(atual);
        atual = rota[atual];
    }

    Collections.reverse(caminho);

    if (caminho.get(0) != origem) {
        System.out.println("Não existe caminho direto de " + origem + " até " + destino);
        return;
    }

    System.out.print("Caminho de " + origem + " até " + destino + ": ");
    for (int i = 0; i < caminho.size(); i++) {
        if (i == caminho.size() - 1) {
            System.out.println(caminho.get(i));
        } else {
            System.out.print(caminho.get(i) + " -> ");
        }
    }
}

```

3. Algoritmo de Bellman-Ford

O Bellman-Ford resolve o mesmo problema que o Dijkstra, mas utiliza a Programação Dinâmica. Ele é mais flexível, projetado para lidar com cenários onde o Dijkstra falha.

Informações Cruciais

- **Aplicação:** Problema de Caminho Mais Curto.
- **Vantagem Principal:** **Suporta arestas com pesos negativos** e é capaz de detectar **ciclos de peso negativo** (onde o peso total do ciclo é menor que zero, o que impossibilita a definição de um caminho mínimo finito).
- **Estratégia:** Relaxa todas as arestas do grafo exatamente $V - 1$ vezes (sendo V o número de vértices). Se no passo V uma distância ainda puder ser reduzida, significa que existe um ciclo negativo.
- **Complexidade de Tempo:** $O(V \cdot E)$, onde V é o número de vértices e E é o número de arestas. É mais lento que o Dijkstra, sendo preferido apenas quando há possibilidade de pesos negativos.

Materiais de apoio:

https://youtu.be/9PHkk0UavIM?si=DB_zxLvckX6wzw2l

<https://youtu.be/obWXjtg0L64?si=Z52IT08Pq9uNB4ev>

https://algorithms.discrete.ma.tum.de/graph-algorithms/spp-bellman-ford/index_en.html (aqui você visualiza cada iteração num desenho, enquanto vê o código)

Implementação em Java

Aqui, a representação mais simples é uma lista plana de arestas. Criamos uma classe estática `Aresta` e iteramos sobre um array simples.

Java

```
public class BellmanFord{

    static class Aresta{
        int origem, destino, peso;
        Aresta(int origem, int destino, int peso) {
            this.origem = origem;
            this.destino = destino;
            this.peso = peso;
        }
    }

    public void bellmanFord(int n, List<List<Vizinho>> graf
```

```

o, int origem) {
    int[] distancias = new int[n];
    int[] rota = new int[n];

    for (int i = 0; i < n; i++) {
        distancias[i] = Integer.MAX_VALUE;
        rota[i] = -1;
    }
    distancias[origem] = 0;

    for (int k = 0; k < n; k++) {
        boolean altera = false;

        for (int u = 0; u < n; u++) {

            if (distancias[u] == Integer.MAX_VALUE)
                continue;

            for (Vizinho vizinho : grafo.get(u)) {
                int v = vizinho.destino;
                int peso = vizinho.peso;

                if (distancias[v] > distancias[u] + pes
o) {

                    distancias[v] = distancias[u] + pes
o;

                    rota[v] = u;
                    altera = true;
                }
            }
        }

        if (!altera) {
            break;
        }

        if (k == n && altera) {
            System.out.println("Ciclo negativo detectad

```

```
o!");
    }
}
}
```

4. Algoritmo de Floyd-Warshall

O algoritmo de Floyd-Warshall é uma solução clássica baseada em programação dinâmica para encontrar caminhos mínimos entre todos os pares de vértices de um grafo de forma simultânea.

Informações Cruciais

- **Aplicação:** Problema de Caminhos Mínimos de Todos para Todos (All-Pairs Shortest Path).
- **Vantagem Principal:** Suporta arcos com pesos negativos. É capaz de identificar a existência de ciclos de custo negativo se, ao final, qualquer valor na diagonal principal da matriz de distâncias for menor do que zero ($l_{ii} < 0$).
- **Estratégia:** Utiliza uma matriz L de dimensões $V \times V$. O algoritmo itera através de um vértice intermediário k (de 1 até n) e compara se o caminho direto entre um par (i, j) é maior do que o desvio passando por k ($l_{ik} + l_{kj}$). Se for, a distância é atualizada (relaxação).
- **Complexidade de Tempo:** $\mathcal{O}(V^3)$ de forma fixa, devido a três laços aninhados que percorrem todos os vértices. É altamente eficiente e simples para grafos densos.

Materiais de apoio:

<https://www.cs.usfca.edu/~galles/visualization/Floyd.html>

Implementação em Java

```
public class FloydWarshall{
    public static final int INF = 999999999; // Representação de infinito

    public void floydWarshall(int n, int[][] matrizPesos){
        int[][] L = new int[n][n];
```

```

        // Inicialização da Matriz L com os pesos diretos d
as arestas
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (i == j) {
                    L[i][j] = 0;
                } else if (matrizPesos[i][j] != 0) {
                    L[i][j] = matrizPesos[i][j];
                } else {
                    L[i][j] = INF;
                }
            }
        }

        for (int k = 0; k < n; k++) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    if (L[i][k] != INF && L[k][j] != INF) {
                        if (L[i][j] > L[i][k] + L[k][j]) {
                            L[i][j] = L[i][k] + L[k][j];
                        }
                    }
                }
            }
        }

        // Verificação de ciclos de custo negativo
        for (int i = 0; i < n; i++) {
            if (L[i][i] < 0) {
                System.out.println("Aviso: Ciclo negativo d
etectado a partir do vértice " + i);
            }
        }
    }
}

```

5. Tabela Comparativa Geral

Característica	Dijkstra	Bellman-Ford	Floyd-Warshall
Paradigma	Guloso (Greedy)	Programação Dinâmica	Programação Dinâmica
Escopo do Resultado	Origem única para todos	Origem única para todos	Todos os pares simultaneamente
Pesos Negativos	Não suporta	Suporta perfeitamente	Suporta perfeitamente
Ciclos Negativos	Não detecta	Detecta e alerta	Detecta e alerta
Complexidade de Tempo	$O(V^2)$ (Array)	$O(V \cdot E)$	$O(V^3)$
Caso de Uso Ideal	Redes de GPS urbanas com custos estritamente positivos.	Grafos esparsos que contenham penalidades negativas ou controle de restrições.	Matrizes completas de distâncias em grafos densos onde caminhos múltiplos importam

6. Questionário

Pergunta 1

Em relação aos algoritmos de busca e percurso em grafos, considere as seguintes afirmações:

- I. A Busca em Profundidade (DFS) utiliza nativamente uma estrutura de dados do tipo Fila (FIFO) para gerenciar os vértices a serem visitados.
- II. Em um grafo não ponderado, a Busca em Largura (BFS) garante encontrar o caminho mínimo (com o menor número de arestas) entre um vértice de origem e qualquer outro vértice alcançável.
- III. A complexidade de tempo de ambos os algoritmos DFS e BFS, quando implementados utilizando uma lista de adjacências para representar um grafo com V vértices e E arestas, é de $O(V+E)$.

Qual ou quais afirmações estão corretas?

Pergunta 2

Um analista de sistemas precisa escolher um algoritmo para solucionar o problema de caminhos mínimos de todos para todos (All-Pairs Shortest Path) em um grafo denso que possui pesos negativos nas arestas. Qual algoritmo

apresenta a abordagem mais direta e a complexidade de tempo adequada para este cenário específico e qual a sua complexidade? Existe algo além a ser considerado?

Pergunta 3

Uma propriedade fundamental da Busca em Largura (BFS) em grafos não direcionados é a capacidade de testar se um grafo é bipartido (ou seja, se pode ter seus vértices coloridos com 2 cores sem que vértices adjacentes compartilhem a mesma cor). Durante a BFS, o grafo será considerado NÃO bipartido se, e somente se:

Pergunta 4

Considere o algoritmo de Floyd-Warshall aplicado a um grafo com n vértices indexados de 0 a $n-1$. O algoritmo utiliza uma matriz de distâncias L , e sua estrutura interna baseia-se em três laços aninhados controlados pelas variáveis k , i , e j . Qual é a função exata da variável do laço mais externo (k) no paradigma de programação dinâmica do algoritmo?

Pergunta 5

Como os algoritmos de Floyd-Warshall e Bellman-Ford podem ser utilizados para detectar de forma robusta a existência de ciclos de custo negativo em um grafo direcionado após a sua conclusão?

Pergunta 6

Suponha que um grafo não direcionado possua pesos positivos e distintos em suas arestas. Se executarmos o algoritmo de Dijkstra e o algoritmo de Floyd-Warshall para encontrar os caminhos mínimos do grafo, o que podemos afirmar sobre os caminhos gerados por ambos os métodos?

Pergunta 7

Um estudante de engenharia de software implementou a Busca em Largura (BFS) esquecendo-se de marcar os vértices como 'visitados' no momento em que eles são inseridos na Fila, fazendo a marcação apenas no instante em que o vértice é removido da fila (desenfileirado). Qual a consequência direta desse erro no comportamento do algoritmo?

Pergunta 8

No contexto do algoritmo de Floyd-Warshall, se desejarmos reconstruir o caminho mínimo propriamente dito entre os vértices i e j (e não apenas saber o custo), costuma-se manter uma matriz auxiliar de predecessores ou

intermediários, denominada Π . Se a relaxação determinar que $L[i][j] > L[i][k] + L[k][j]$, como deve ser atualizada a entrada correspondente $\Pi[i][j]$?

- A. $\Pi[i][j] = \Pi[k][j]$ (ou k , a depender se rastreamos o predecessor direto ou o ponto de quebra).
- B. $\Pi[i][j] = -1$
- C. $\Pi[i][j] = \Pi[i][k] + \Pi[k][j]$
- D. $\Pi[i][j] = \Pi[j][i]$